# Advanced machine learning and data analysis for the physical sciences

Morten Hjorth-Jensen[1]

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway[1]

April 16, 2026

© 1999-2026, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

# Plans for the week April 13-17, 2026

## Generative methods, energy models and Boltzmann machines

- Summary of discussions on Restricted Boltzmann machines, reminder from last week
- Gibbs sampling and Markov chains
- Discussions of various Boltzmann machines
- Implementation of simple restricted Boltzmann machines, own codes and using Pytorch
- Introduction to Variational Autoencoders (VAEs), basic mathematical formalism

# Reading recommendations

- Boltzmann machines: Goodfellow et al chapters 18.1-18.2, 20.1-20-7; To create Boltzmann machine using Keras, see Babcock and Bali chapter 4, see https://github.com/PacktPublishing/Hands-On-Generative-AI-with-Python-and-TensorFlow-2/blob/master/Chapter_4/models/rbm.py
- More on Boltzmann machines: see also Foster, chapter 7 on energy-based models at https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition/tree/main/notebooks/07_ebm/01_ebm
- VAEs: Goodfellow et al, for VAEs see sections 20.10-20.11

# Essential elements of generative models, from last week

The aim of generative methods is to train a probability distribution $p$. The methods we will focus on are:

1. Energy based models, with the family of Boltzmann distributions as a typical example
2. Variational autoencoders, based on our discussions on autoencoders
3. Generative adversarial networks (GANs) and
4. Diffusion models

# Energy models, reminders from last two lectures

During the last two weeks we defined a domain $\boldsymbol{X}$ of stochastic variables $\boldsymbol{X} = \{x_0, x_1, \dots, x_{n-1}\}$ with a pertinent probability distribution

$$p(\boldsymbol{X}) = \prod_{x_i \in \boldsymbol{X}} p(x_i),$$

where we have assumed that the random variables $x_i$ are all independent and identically distributed (iid).
We will now assume that we can defined this function in terms of optimization parameters $\boldsymbol{\Theta}$, which could be the biases and weights of deep network, and a set of hidden variables we also assume to be random variables which also are iid. The domain of these variables is $\boldsymbol{H} = \{h_0, h_1, \dots, h_{m-1}\}$.

# Probability model

We define a probability

$$p(x_i, h_j; \boldsymbol{\Theta}) = \frac{f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

where $f(x_i, h_j; \boldsymbol{\Theta})$ is a function which we assume is larger or equal than zero and obeys all properties required for a probability distribution and $Z(\boldsymbol{\Theta})$ is a normalization constant. Inspired by statistical mechanics, we call it often for the partition function. It is defined as (assuming that we have discrete probability distributions)

$$Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}).$$

# Marginal and conditional probabilities

We can in turn define the marginal probabilities

$$p(x_i; \boldsymbol{\Theta}) = \frac{\sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

and

$$p(h_i; \boldsymbol{\Theta}) = \frac{\sum_{x_i \in \boldsymbol{X}} f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})}.$$

# Change of notation

**Note the change to a vector notation**. A variable like $\boldsymbol{x}$ represents now a specific **configuration**. We can generate an infinity of such configurations. The final partition function is then the sum over all such possible configurations, that is

$$Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}),$$

changes to

$$Z(\boldsymbol{\Theta}) = \sum_{\boldsymbol{x}} \sum_{\boldsymbol{h}} f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}).$$

If we have a binary set of variable $x_i$ and $h_j$ and $M$ values of $x_i$ and $N$ values of $h_j$ we have in total $2^M$ and $2^N$ possible $\boldsymbol{x}$ and $\boldsymbol{h}$ configurations, respectively.
We see that even for the modest binary case, we can easily approach a number of configuration which is not possible to deal with.

# Optimization problem

At the end, we are not interested in the probabilities of the hidden variables. The probability we thus want to optimize is

$$p(\boldsymbol{X};\boldsymbol{\Theta}) = \prod_{x_i \in \boldsymbol{X}} p(x_i;\boldsymbol{\Theta}) = \prod_{x_i \in \boldsymbol{X}} \left( \frac{\sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})} \right),$$

which we rewrite as

$$p(\boldsymbol{X};\boldsymbol{\Theta}) = \frac{1}{Z(\boldsymbol{\Theta})} \prod_{x_i \in \boldsymbol{X}} \left( \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}) \right).$$

# Further simplifications

We simplify further by rewriting it as

$$p(\boldsymbol{X};\boldsymbol{\Theta}) = \frac{1}{Z(\boldsymbol{\Theta})} \prod_{x_i \in \boldsymbol{X}} f(x_i;\boldsymbol{\Theta}),$$

where we used $p(x_i;\boldsymbol{\Theta}) = \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j;\boldsymbol{\Theta})$. The optimization problem is then

$$\arg \max_{\boldsymbol{\Theta} \in \mathbb{R}^p} p(\boldsymbol{X};\boldsymbol{\Theta}).$$

# Optimizing the logarithm instead

Computing the derivatives with respect to the parameters $\boldsymbol{\Theta}$ is easier (and equivalent) with taking the logarithm of the probability. We will thus optimize

$$\arg \max_{\boldsymbol{\Theta} \in \mathbb{R}^p} \log p(\boldsymbol{X}; \boldsymbol{\Theta}),$$

which leads to

$$\nabla_{\boldsymbol{\Theta}} \log p(\boldsymbol{X}; \boldsymbol{\Theta}) = 0.$$

# Expression for the gradients

This leads to the following equation

$$\nabla_{\boldsymbol{\Theta}} \log p(\boldsymbol{X}; \boldsymbol{\Theta}) = \nabla_{\boldsymbol{\Theta}} \left( \sum_{x_i \in \boldsymbol{X}} \log f(x_i; \boldsymbol{\Theta}) \right) - \nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = 0.$$

The first term is called the positive phase and we assume that we have a model for the function $f$ from which we can sample values. Below we will develop an explicit model for this. The second term is called the negative phase and is the one which leads to more difficulties.

# The derivative of the partition function

The partition function, defined above as

$$Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}),$$

is in general the most problematic term. In principle both $x$ and $h$ can span large degrees of freedom, if not even infinitely many ones, and computing the partition function itself is often not desirable or even feasible. The above derivative of the partition function can however be written in terms of an expectation value which is in turn evaluated using Monte Carlo sampling and the theory of Markov chains, popularly shortened to MCMC (or just $\mathrm{MC}^2$).

# Explicit expression for the derivative

We can rewrite

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \frac{\nabla_{\boldsymbol{\Theta}} Z(\boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

which reads in more detail

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \frac{\nabla_{\boldsymbol{\Theta}} \sum_{x_i \in \boldsymbol{X}} f(x_i; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})}.$$

We can rewrite the function $f$ (we have assumed that is larger or equal than zero) as $f = \exp \log f$. We can then reqrite the last equation as

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \frac{\sum_{x_i \in \boldsymbol{X}} \nabla_{\boldsymbol{\Theta}} \exp \log f(x_i; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})}.$$

# Final expression

Taking the derivative gives us

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \frac{\sum_{x_i \in \boldsymbol{X}} f(x_i; \boldsymbol{\Theta}) \nabla_{\boldsymbol{\Theta}} \log f(x_i; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

which is the expectation value of $\log f$

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} p(x_i; \boldsymbol{\Theta}) \nabla_{\boldsymbol{\Theta}} \log f(x_i; \boldsymbol{\Theta}),$$

that is

$$\nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = \mathbb{E}(\log f(x_i; \boldsymbol{\Theta})).$$

This quantity is evaluated using Monte Carlo sampling, with Gibbs sampling as the standard sampling rule. Before we discuss the explicit algorithms, we need to remind ourselves about Markov chains and sampling rules like the Metropolis-Hastings algorithm and Gibbs sampling.

# Positive and negative phases

As discussed earlier, the data-dependent term in the gradient is known as the positive phase of the gradient, while the model-dependent term is known as the negative phase of the gradient. The aim of the training is to lower the energy of configurations that are near observed data points (increasing their probability), and raising the energy of configurations that are far from observed data points (decreasing their probability).

# Reminder on Markov Chain Monte Carlo, reminder from last week

- We want to study a physical system which evolves towards equilibrium, from given initial conditions.
- We start with a PDF $w(x_0, t_0)$ and we want to understand how the system evolves with time.
- We want to reach a situation where after a given number of time steps we obtain a steady state. This means that the system reaches its most likely state (equilibrium situation)
- Our PDF is normally a multidimensional object whose normalization constant is impossible to find.
- Analytical calculations from $w(x, t)$ are not possible.
- To sample directly from from $w(x, t)$ is not possible/difficult.
- The transition probability $W$ is also not known.
- How can we establish that we have reached a steady state? Sounds impossible!

**Use Markov chain Monte Carlo**

# Brownian motion and Markov processes

A Markov process is a random walk with a selected probability for making a move. The new move is independent of the previous history of the system.
The Markov process is used repeatedly in Monte Carlo simulations in order to generate new random states.
The reason for choosing a Markov process is that when it is run for a long enough time starting with a random state, we will eventually reach the most likely state of the system.
In thermodynamics, this means that after a certain number of Markov processes we reach an equilibrium distribution.
This mimicks the way a real system reaches its most likely state at a given temperature of the surroundings.

# Brownian motion and Markov processes, Ergodicity and Detailed balance

To reach this distribution, the Markov process needs to obey two important conditions, that of **ergodicity** and **detailed balance**. These conditions impose then constraints on our algorithms for accepting or rejecting new random states.
The Metropolis algorithm discussed here abides to both these constraints.
The Metropolis algorithm is widely used in Monte Carlo simulations and the understanding of it rests within the interpretation of random walks and Markov processes.
For a proof the ergodic theorem see
https://www.pnas.org/doi/10.1073/pnas.17.2.656.

# Brownian motion and Markov processes, jargon

In a random walk one defines a mathematical entity called a **walker**, whose attributes completely define the state of the system in question.
The state of the system can refer to any physical quantities, from the vibrational state of a molecule specified by a set of quantum numbers, to the brands of coffee in your favourite supermarket.
The walker moves in an appropriate state space by a combination of deterministic and random displacements from its previous position.
This sequence of steps forms a **chain**.

# Brownian motion and Markov processes, sequence of ingredients

- We want to study a physical system which evolves towards equilibrium, from given initial conditions.
- Markov chains are intimately linked with the physical process of diffusion.
- From a Markov chain we can then derive the conditions for detailed balance and ergodicity. These are the conditions needed for obtaining a steady state.
- The widely used algorithm for doing this is the so-called Metropolis algorithm, in its refined form the Metropolis-Hastings algorithm.

# Markov processes

A Markov process allows in principle for a microscopic description of Brownian motion. As with the random walk studied in the previous section, we consider a particle which moves along the $x$-axis in the form of a series of jumps with step length $\Delta x = l$. Time and space are discretized and the subsequent moves are statistically independent, i.e., the new move depends only on the previous step and not on the results from earlier trials. We start at a position $x = jl = j\Delta x$ and move to a new position $x = i\Delta x$ during a step $\Delta t = \epsilon$, where $i \geq 0$ and $j \geq 0$ are integers. The original probability distribution function (PDF) of the particles is given by $p_i(t = 0)$ where $i$ refers to a specific position on the grid in

The function $p_i(t = 0)$ is now the discretized version of $p(x, t)$. We can regard the discretized PDF as a vector.

# Markov processes

For the Markov process we have a transition probability from a position $x = jl$ to a position $x = il$ given by

$$W_{ij}(\epsilon) = W(il - jl, \epsilon) = \begin{cases} \frac{1}{2} & |i-j| = 1 \\ 0 & \text{else} \end{cases},$$

where $W_{ij}$ is normally called the transition probability and we can represent it, see below, as a matrix. **Here we have specialized to a case where the transition probability is known**.
Our new PDF $p_i(t = \epsilon)$ is now related to the PDF at $t = 0$ through the relation

$$p_i(t = \epsilon) = \sum_j W(j \to i) p_j(t = 0).$$

This equation represents the discretized time-development of an original PDF with equal probability of jumping left or right.

# Markov processes, the probabilities

Since both $\boldsymbol{W}$ and $\boldsymbol{p}$ represent probabilities, they have to be normalized, i.e., we require that at each time step we have

$$\sum_i p_i(t) = 1,$$

and

$$\sum_j W(j \to i) = 1,$$

which applies for all $j$-values. The further constraints are $0 \leq W_{ij} \leq 1$ and $0 \leq p_j \leq 1$. Note that the probability for remaining at the same place is in general not necessarily equal zero.

# Markov processes

The time development of our initial PDF can now be represented through the action of the transition probability matrix applied $n$ times. At a time $t_n = n\epsilon$ our initial distribution has developed into

$$p_i(t_n) = \sum_j W_{ij}(t_n) p_j(0),$$

and defining

$$W(il - jl, n\epsilon) = (W^n(\epsilon))_{ij}$$

we obtain

$$p_i(n\epsilon) = \sum_j (W^n(\epsilon))_{ij} p_j(0),$$

or in matrix-vector form

$$\boldsymbol{p}(n\epsilon) = \boldsymbol{W}^n(\epsilon)\boldsymbol{p}(0). \tag{1}$$

# What do the results mean?

We have after $t$-steps

$$\boldsymbol{p}(t) = \boldsymbol{W}^t \boldsymbol{p}(0),$$

with $\boldsymbol{p}(0)$ the distribution at $t = 0$ and $\boldsymbol{W}$ representing the transition probability matrix.

# Understanding the basics

We can always expand $\boldsymbol{w}(0)$ in terms of the right eigenvectors $\boldsymbol{v}$ of $\boldsymbol{W}$ as

$$\boldsymbol{p}(0) = \sum_i \alpha_i \boldsymbol{v}_i,$$

resulting in

$$\boldsymbol{p}(t) = \boldsymbol{W}^t \boldsymbol{p}(0) = \boldsymbol{W}^t \sum_i \alpha_i \boldsymbol{v}_i = \sum_i \lambda_i^t \alpha_i \boldsymbol{v}_i,$$

with $\lambda_i$ the $i^{\text{th}}$ eigenvalue corresponding to the eigenvector $\boldsymbol{v}_i$. If we assume that $\lambda_0$ is the largest eigenvector we see that in the limit $t \to \infty$, $\boldsymbol{p}(t)$ becomes proportional to the corresponding eigenvector $\boldsymbol{v}_0$. This is our steady state or final distribution.

# Basics of the Metropolis Algorithm

The Metropolis algorithm is a method to sample a normalized probability distribution by a stochastic process. We define $p_i^{(n)}$ to be the probability for finding the system in the state $i$ at step $n$. In the simulations, our assumption is that we have a model for $p_i^{(n)}$, but we do not know $W$. We will hence model $W$ in terms of a likelihood for making transition $T$ and a likelihood for accepting a transition. That is

$$W_{i\to j} = A_{i\to j} T_{i\to j}$$

# The basic of the Metropolis Algorithm

- Sample a possible new state $j$ with some probability $T_{i\to j}$.
- Accept the new state $j$ with probability $A_{i\to j}$ and use it as the next sample.
- With probability $1 - A_{i\to j}$ the move is rejected and the original state $i$ is used again as a sample.

We wish to derive the required properties of $T$ and $A$ such that $p_i^{(n\to\infty)} \to p_i$ so that starting from any distribution, the method converges to the correct distribution. Note that the description here is for a discrete probability distribution. Replacing probabilities $p_i$ with expressions like $p(x_i)dx_i$ will take all of these over to the corresponding continuum expressions.

# More on the Metropolis

The dynamical equation for $p_i^{(n)}$ can be written directly from the description above. The probability of being in the state $i$ at step $n$ is given by the probability of being in any state $j$ at the previous step, and making an accepted transition to $i$ added to the probability of being in the state $i$, making a transition to any state $j$ and rejecting the move:

$$p_i^{(n)} = \sum_j \left[ p_j^{(n-1)} T_{j\to i} A_{j\to i} + p_i^{(n-1)} T_{i\to j} \left(1 - A_{i\to j}\right) \right] . \qquad (2)$$

# Metropolis algorithm, setting it up

Since the probability of making some transition must be 1, $\sum_j T_{i\to j} = 1$, and Eq. (2) becomes

$$p_i^{(n)} = p_i^{(n-1)} + \sum_j \left[ p_j^{(n-1)} T_{j\to i} A_{j\to i} - p_i^{(n-1)} T_{i\to j} A_{i\to j} \right] . \quad (3)$$

# Metropolis continues

For large $n$ we require that $p_i^{(n\to\infty)} = p_i$, the desired probability distribution. Taking this limit, gives the balance requirement

$$\sum_j \left[p_j T_{j\to i} A_{j\to i} - p_i T_{i\to j} A_{i\to j}\right] = 0, \tag{4}$$

# Detailed Balance

The balance requirement is very weak. Typically the much stronger detailed balance requirement is enforced, that is rather than the sum being set to zero, we set each term separately to zero and use this to determine the acceptance probabilities. Rearranging, the result is

$$\frac{A_{j\to i}}{A_{i\to j}} = \frac{p_i\, T_{i\to j}}{p_j\, T_{j\to i}}\,. \tag{5}$$

This is the detailed balance requirement

# More on Detailed Balance

The Metropolis choice is to maximize the $A$ values, that is

$$A_{j\to i} = \min\left(1, \frac{p_i\, T_{i\to j}}{p_j\, T_{j\to i}}\right). \tag{6}$$

Other choices are possible, but they all correspond to multiplying $A_{i\to j}$ and $A_{j\to i}$ by the same constant smaller than unity. The penalty function method uses just such a factor to compensate for $p_i$ that are evaluated stochastically and are therefore noisy. Having chosen the acceptance probabilities, we have guaranteed that if the $p_i^{(n)}$ has equilibrated, that is if it is equal to $p_i$, it will remain equilibrated.

# Gibbs sampling

An efficient way if performing the sampling is through the use of Gibbs sampling. The latter uses the conditional probability instead of the full probability as done in the Metropolis algorithm.
Gibbs sampling is useful for sampling from high-dimensional distributions where single-variable conditional distributions are known.
For example, say it is too expensive to sample from $p(x_0, x_1, x_2, ..., x_d)$. With Gibbs sampling, we initialize all variables to arbitrary values. Then while taking each sample, we also iterate through the dimensions and replace its value with a sample from the univariate conditional distribution. For example we can update $x_1$ using $p(x_1 \mid x_0, x_2, ..., x_d)$, which is easy to sample over because it is only one dimension.

# Understanding Gibbs samoling

This part is best seen with the jupyter-notebook. These notes have been adapted from https://www.inf.ed.ac.uk/teaching/courses/mlpr/2017/

To illustrate Gibbs sampling we will sample several points and compare them with those generated from a known distribution, in our case the well-known two-dimensional Gaussian defined as

$$p(a,b) = \mathcal{N}\left(\begin{bmatrix} a \\ b \end{bmatrix}; \begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \begin{bmatrix} \sigma_a & cov(a,b) \\ cov(a,b) & \sigma_b \end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix} a \\ b \end{bmatrix}; \right.$$

```
import numpy as np
import matplotlib.pyplot as plt
# two dimensions
D = 2

# set up the means (standard normal distribution
a_mu = 0
b_mu = 0
# and the variances and covariances
a_sigma = 1
b_sigma = 1
a_b_cov = 0.5

joint_cov = np.vstack(((a_sigma, a_b_cov), (a_b_cov, b_sigma)))
```

# Conditional probabilities

Gibbs sampling requires conditional distributions for each variable.

In the case of Gaussians, there is a closed-form for the conditional.

With a multivariate Gaussian we have

$$p(\mathbf{f}, \mathbf{g}) = \mathcal{N}\left(\begin{bmatrix} \mathbf{f} \\ \mathbf{g} \end{bmatrix}; \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix}, \begin{bmatrix} A & C^\top \\ C & B \end{bmatrix}\right),$$

with the conditional probability

$$p(\mathbf{f} \mid \mathbf{g}) = \mathcal{N}(\mathbf{f}; \mathbf{a} + CB^{-1}(\mathbf{g} - \mathbf{b}), A - CB^{-1}C^\top).$$

```
def get_conditional_dist(joint_mu, joint_cov, var_index):
    '''Returns the conditional distribution given the joint distributi
    the conditional probability should use.
    Right now this only works for 2-variable joint distributions.

      joint_mu: joint distribution's mu
      joint_cov: joint distribution's covariance
      var_index: index of the variable in the joint distribution. Ever
        conditioned on. For example, if the joint distribution p(a, b,
        to get p(c | a, b), use var_index = 2.

    returns:
      a function that can sample from the univariate conditional distr
    '''
    assert joint_mu.shape[0] == 2, 'Sorry, this function only works fo
    a = joint_mu[var_index]
```

# Boltzmann Machines, marginal and conditional probabilities

A generative model can learn to represent and sample from a probability distribution. The core idea is to learn a parametric model of the probability distribution from which the training data was drawn. As an example

1. A model for images could learn to draw new examples of cats and dogs, given a training dataset of images of cats and dogs.
2. Generate a sample of an ordered or disordered Ising model phase, having been given samples of such phases.
3. Model the trial function for Monte Carlo calculations

# Generative and discriminative models

Generative and discriminative models use both gradient-descent based learning procedures for minimizing cost functions
However, in energy based models we don't use backpropagation and automatic differentiation for computing gradients, instead we turn to Markov Chain Monte Carlo methods.
A typical deep neural network has several hidden layers. A restricted Boltzmann machine has normally one hidden layer, however several RBMs can be stacked to make up deep Belief Networks, of which they constitute the building blocks.

# Basics of the Boltzmann machine

A BM is what we would call an undirected probabilistic graphical model with stochastic continuous or discrete units.

It is interpreted as a stochastic recurrent neural network where the state of each unit(neurons/nodes) depends on the units it is connected to. The weights in the network represent thus the strength of the interaction between various units/nodes.

# More about the basics

A standard BM network is divided into a set of observable and visible units $\boldsymbol{x}$ and a set of unknown hidden units/nodes $\boldsymbol{h}$.

Additionally there can be bias nodes for the hidden and visible layers. These biases are normally set to 1.

BMs are stackable, meaning they cwe can train a BM which serves as input to another BM. We can construct deep networks for learning complex PDFs. The layers can be trained one after another, a feature which makes them popular in deep learning

# Difficult to train

However, they are often hard to train. This leads to the introduction of so-called restricted BMs, or RBMS. Here we take away all lateral connections between nodes in the visible layer as well as connections between nodes in the hidden layer.

# The network layers

1. A function $\boldsymbol{x}$ that represents the visible layer, a vector of $M$ elements (nodes). This layer represents both what the RBM might be given as training input, and what we want it to be able to reconstruct. This might for example be the pixels of an image, the spin values of the Ising model, or coefficients representing speech.
2. The function $\boldsymbol{h}$ represents the hidden, or latent, layer. A vector of $N$ elements (nodes). Also called "feature detectors".

# Goal of hidden layer

The goal of the hidden layer is to increase the model's expressive power. We encode complex interactions between visible variables by introducing additional, hidden variables that interact with visible degrees of freedom in a simple manner, yet still reproduce the complex correlations between visible degrees in the data once marginalized over (integrated out).

# The parameters

The network parameters, to be optimized/learned:

1. $\boldsymbol{a}$ represents the visible bias, a vector of same length $M$ as $\boldsymbol{x}$.
2. $\boldsymbol{b}$ represents the hidden bias, a vector of same length $N$ as $\boldsymbol{h}$.
3. $\boldsymbol{W}$ represents the interaction weights, a matrix of size $M \times N$.

Note that we have specified the lengths of $bmx$ and $\boldsymbol{h}$. These lengths define the number of visible and hidden units, respectively.

# Joint distribution

The restricted Boltzmann machine is described by a Boltzmann distribution

$$P_{rbm}(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta}) = \frac{1}{Z(\boldsymbol{\Theta})} \exp -(E(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta})),$$

where $Z$ is the normalization constant or partition function discussed earlier and defined as

$$Z(\boldsymbol{\Theta}) = \int \int \exp -E(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta}) d\boldsymbol{x} d\boldsymbol{h}.$$

It is common to set the temperature $T$ to one. It is omitted in the equations above. The energy is thus a dimensionless function.

# Network Elements, the energy function

The function $E(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta})$ gives the **energy** of a configuration (pair of vectors) $(\boldsymbol{x}, \boldsymbol{h})$. The lower the energy of a configuration, the higher the probability of it. This function also depends on the parameters $\boldsymbol{a}$, $\boldsymbol{b}$ and $W$. Thus, when we adjust them during the learning procedure, we are adjusting the energy function to best fit our problem.

# Defining different types of RBMs

There are different variants of RBMs, and the differences lie in the types of visible and hidden units we choose as well as in the implementation of the energy function $E(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta})$. The connection between the nodes in the two layers is given by the weights $w_{ij}$.

## Binary-Binary RBM:

RBMs were first developed using binary units in both the visible and hidden layer. The corresponding energy function is defined as follows:

$$E(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta}) = -\sum_{i}^{M} x_i a_i - \sum_{j}^{N} b_j h_j - \sum_{i,j}^{M,N} x_i w_{ij} h_j,$$

where the binary values taken on by the nodes are most commonly 0 and 1.

# Gaussian-binary RBM

Another varient is the RBM where the visible units are Gaussian while the hidden units remain binary:

$$E(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta}) = \sum_{i}^{M} \frac{(x_i - a_i)^2}{2\sigma_i^2} - \sum_{j}^{N} b_j h_j - \sum_{i,j}^{M,N} \frac{x_i w_{ij} h_j}{\sigma_i^2}.$$

This type of RBMs are useful when we model continuous data (i.e., we wish $\boldsymbol{x}$ to be continuous). The paramater $\sigma_i^2$ is meant to represent a variance and is foten just set to one.

# Cost function

When working with a training dataset, the most common training approach is maximizing the log-likelihood of the training data. The log likelihood characterizes the log-probability of generating the observed data using our generative model. Using this method our cost function is chosen as the negative log-likelihood. The learning then consists of trying to find parameters that maximize the probability of the dataset, and is known as Maximum Likelihood Estimation (MLE).
Denoting the parameters as $\mathbf{\Theta} = a_1, ..., a_M, b_1, ..., b_N, w_{11}, ..., w_{MN}$, the log-likelihood is given by

$$\begin{aligned}\mathcal{L}(\{\Theta_i\}) &= \langle \log P_\theta(\boldsymbol{x}) \rangle_{data} \\ &= -\langle E(\boldsymbol{x}; \{\Theta_i\}) \rangle_{data} - \log Z(\{\Theta_i\}),\end{aligned}$$

where we used that the normalization constant does not depend on the data, $\langle \log Z(\{\Theta_i\}) \rangle = \log Z(\{\Theta_i\})$ Our cost function is the negative log-likelihood, $\mathcal{C}(\{\Theta_i\}) = -\mathcal{L}(\{\Theta_i\})$

# Optimization / Training

The training procedure of choice often is Stochastic Gradient Descent (SGD). It consists of a series of iterations where we update the parameters according to the equation

$$\mathbf{\Theta}_{k+1} = \mathbf{\Theta}_k - \eta \nabla \mathcal{C}(\mathbf{\Theta}_k)$$

at each $k$-th iteration. There are a range of variants of the algorithm which aim at making the learning rate $\eta$ more adaptive so the method might be more efficient while remaining stable.

# Gradients

We now need the gradient of the cost function in order to minimize it. We find that

$$\begin{aligned}\frac{\partial \mathcal{C}(\{\Theta_i\})}{\partial \Theta_i} &= \langle \frac{\partial E(\boldsymbol{x};\Theta_i)}{\partial \Theta_i} \rangle_{data} + \frac{\partial \log Z(\{\Theta_i\})}{\partial \Theta_i} \\ &= \langle O_i(\boldsymbol{x}) \rangle_{data} - \langle O_i(\boldsymbol{x}) \rangle_{model}.\end{aligned}$$

# Simplifications

In order to simplify notation we defined the "operator"

$$O_i(\boldsymbol{x}) = \frac{\partial E(\boldsymbol{x}; \Theta_i)}{\partial \Theta_i},$$

and used the statistical mechanics relationship between expectation values and the log-partition function:

$$\langle O_i(\boldsymbol{x}) \rangle_{model} = \mathrm{Tr} P_\Theta(\boldsymbol{x}) O_i(\boldsymbol{x}) = -\frac{\partial \log Z(\{\Theta_i\})}{\partial \Theta_i}.$$

# Positive and negative phases

As discussed earlier, the data-dependent term in the gradient is known as the positive phase of the gradient, while the model-dependent term is known as the negative phase of the gradient. The aim of the training is to lower the energy of configurations that are near observed data points (increasing their probability), and raising the energy of configurations that are far from observed data points (decreasing their probability).

# Gradient examples

The gradient of the negative log-likelihood cost function of a Binary-Binary RBM is then

$$\begin{aligned}
\frac{\partial \mathcal{C}(w_{ij}, a_i, b_j)}{\partial w_{ij}} &= \langle x_i h_j \rangle_{data} - \langle x_i h_j \rangle_{model} \\
\frac{\partial \mathcal{C}(w_{ij}, a_i, b_j)}{\partial a_{ij}} &= \langle x_i \rangle_{data} - \langle x_i \rangle_{model} \\
\frac{\partial \mathcal{C}(w_{ij}, a_i, b_j)}{\partial b_{ij}} &= \langle h_i \rangle_{data} - \langle h_i \rangle_{model}.
\end{aligned}$$

To get the expectation values with respect to the *data*, we set the visible units to each of the observed samples in the training data, then update the hidden units according to the conditional probability found before. We then average over all samples in the training data to calculate expectation values with respect to the data.

# Kullback-Leibler relative entropy

When the goal of the training is to approximate a probability distribution, as it is in generative modeling, another relevant measure is the **Kullback-Leibler divergence**, also known as the relative entropy or Shannon entropy. It is a non-symmetric measure of the dissimilarity between two probability density functions $p$ and $q$. If $p$ is the unkown probability which we approximate with $q$, we can measure the difference by

$$\mathrm{KL}(p||q) = \int_{-\infty}^{\infty} p(\boldsymbol{x}) \log \frac{p(\boldsymbol{x})}{q(\boldsymbol{x})} d\boldsymbol{x}.$$

# Kullback-Leibler divergence

Thus, the Kullback-Leibler divergence between the distribution of the training data $f(\boldsymbol{x})$ and the model distribution $p(\boldsymbol{x}|\boldsymbol{\Theta})$ is

$$
\begin{aligned}
\mathrm{KL}(f(\boldsymbol{x})||p(\boldsymbol{x}|\boldsymbol{\Theta})) &= \int_{-\infty}^{\infty} f(\boldsymbol{x}) \log \frac{f(\boldsymbol{x})}{p(\boldsymbol{x}|\boldsymbol{\Theta})} d\boldsymbol{x} \\
&= \int_{-\infty}^{\infty} f(\boldsymbol{x}) \log f(\boldsymbol{x}) d\boldsymbol{x} - \int_{-\infty}^{\infty} f(\boldsymbol{x}) \log p(\boldsymbol{x}|\boldsymbol{\Theta}) d\boldsymbol{x} \\
&= \langle \log f(\boldsymbol{x}) \rangle_{f(\boldsymbol{x})} - \langle \log p(\boldsymbol{x}|\boldsymbol{\Theta}) \rangle_{f(\boldsymbol{x})} \\
&= \langle \log f(\boldsymbol{x}) \rangle_{data} + \langle E(\boldsymbol{x}) \rangle_{data} + \log Z \\
&= \langle \log f(\boldsymbol{x}) \rangle_{data} + \mathcal{C}_{LL}.
\end{aligned}
$$

# Maximizing log-likelihood

The first term is constant with respect to $\boldsymbol{\Theta}$ since $f(\boldsymbol{x})$ is independent of $\boldsymbol{\Theta}$. Thus the Kullback-Leibler Divergence is minimal when the second term is minimal. The second term is the log-likelihood cost function, hence minimizing the Kullback-Leibler divergence is equivalent to maximizing the log-likelihood.
To further understand generative models it is useful to study the gradient of the cost function which is needed in order to minimize it using methods like stochastic gradient descent.

# More on the partition function

The partition function is the generating function of expectation values, in particular there are mathematical relationships between expectation values and the log-partition function. In this case we have

$$\langle \frac{\partial E(\boldsymbol{x};\Theta_i)}{\partial \Theta_i} \rangle_{model} = \int p(\boldsymbol{x}|\boldsymbol{\Theta}) \frac{\partial E(\boldsymbol{x};\Theta_i)}{\partial \Theta_i} d\boldsymbol{x} = -\frac{\partial \log Z(\Theta_i)}{\partial \Theta_i}.$$

Here $\langle \cdot \rangle_{model}$ is the expectation value over the model probability distribution $p(\boldsymbol{x}|\boldsymbol{\Theta})$.

# Setting up for gradient descent calculations

Using the previous relationship we can express the gradient of the cost function as

$$\begin{aligned}\frac{\partial \mathcal{C}_{LL}}{\partial \Theta_i} =& \langle \frac{\partial E(\boldsymbol{x}; \Theta_i)}{\partial \Theta_i} \rangle_{data} + \frac{\partial \log Z(\Theta_i)}{\partial \Theta_i} \\ =& \langle \frac{\partial E(\boldsymbol{x}; \Theta_i)}{\partial \Theta_i} \rangle_{data} - \langle \frac{\partial E(\boldsymbol{x}; \Theta_i)}{\partial \Theta_i} \rangle_{model}\end{aligned}$$

# Difference of moments

This expression shows that the gradient of the log-likelihood cost function is a **difference of moments**, with one calculated from the data and one calculated from the model. The data-dependent term is called the **positive phase** and the model-dependent term is called the **negative phase** of the gradient. We see now that minimizing the cost function results in lowering the energy of configurations $\boldsymbol{x}$ near points in the training data and increasing the energy of configurations not observed in the training data. That means we increase the model's probability of configurations similar to those in the training data.

# More observations

The gradient of the cost function also demonstrates why gradients of unsupervised, generative models must be computed differently from for those of for example FNNs. While the data-dependent expectation value is easily calculated based on the samples $\boldsymbol{x}_i$ in the training data, we must sample from the model in order to generate samples from which to caclulate the model-dependent term. We sample from the model by using MCMC-based methods. We can not sample from the model directly because the partition function $Z$ is generally intractable.

# Adding hyperparameters

As in supervised machine learning problems, the goal is also here to perform well on **unseen** data, that is to have good generalization from the training data. The distribution $f(x)$ we approximate is not the **true** distribution we wish to estimate, it is limited to the training data. Hence, in unsupervised training as well it is important to prevent overfitting to the training data. Thus it is common to add regularizers to the cost function in the same manner as we discussed for say linear regression.

# Mathematical details

Because we are restricted to potential functions which are positive it is convenient to express them as exponentials.
The original RBM had binary visible and hidden nodes. They were showned to be universal approximators of discrete distributions. It was also shown that adding hidden units yields strictly improved modelling power.

# Binary-binary (BB) RBMs

The common choice of binary values are 0 and 1. However, in some physics applications, -1 and 1 might be a more natural choice. We will here use 0 and 1. We habe the energy function

$$E_{BB}(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta}) = -\sum_{i}^{M} x_i a_i - \sum_{j}^{N} b_j h_j - \sum_{i,j}^{M,N} x_i w_{ij} h_j.$$

# Marginal probability

We have the binary-binary marginal probability defined as

$$
\begin{aligned}
p_{BB}(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta}) &= \frac{1}{Z_{BB}(\boldsymbol{\Theta})} e^{\sum_i^M a_i x_i + \sum_j^N b_j h_j + \sum_{ij}^{M,N} x_i w_{ij} h_j} \\
&= \frac{1}{Z_{BB}(\boldsymbol{\Theta})} e^{\boldsymbol{a}^T \boldsymbol{x} + \boldsymbol{b}^T \boldsymbol{h} + \boldsymbol{x}^T \boldsymbol{W} \boldsymbol{h}}
\end{aligned}
$$

with the partition function

$$
Z_{BB}(\boldsymbol{\Theta}) = \sum_{\boldsymbol{x}, \boldsymbol{h}} e^{\boldsymbol{a}^T \boldsymbol{x} + \boldsymbol{b}^T \boldsymbol{h} + \boldsymbol{x}^T \boldsymbol{W} \boldsymbol{h}}.
$$

# Marginal Probability Density Function for the visible units

In order to find the probability of any configuration of the visible units we derive the marginal probability density function.

$$
\begin{aligned}
p_{BB}(\boldsymbol{x}, \boldsymbol{\Theta}) &= \sum_{\boldsymbol{h}} p_{BB}(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta}) \\
&= \frac{1}{Z_{BB}} \sum_{\boldsymbol{h}} e^{\boldsymbol{a}^T \boldsymbol{x} + \boldsymbol{b}^T \boldsymbol{h} + \boldsymbol{x}^T \boldsymbol{W} \boldsymbol{h}} \\
&= \frac{1}{Z_{BB}} e^{\boldsymbol{a}^T \boldsymbol{x}} \sum_{\boldsymbol{h}} e^{\sum_j^N (b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}) h_j} \\
&= \frac{1}{Z_{BB}} e^{\boldsymbol{a}^T \boldsymbol{x}} \sum_{\boldsymbol{h}} \prod_j^N e^{(b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}) h_j} \\
&= \frac{1}{Z_{BB}} e^{\boldsymbol{a}^T \boldsymbol{x}} \left( \sum_{h_1} e^{(b_1 + \boldsymbol{x}^T \boldsymbol{w}_{*1}) h_1} \times \sum_{h_2} e^{(b_2 + \boldsymbol{x}^T \boldsymbol{w}_{*2}) h_2} \times \right. \\
&\quad \left. \ldots \times \sum_{h_2} e^{(b_N + \boldsymbol{x}^T \boldsymbol{w}_{*N}) h_N} \right)
\end{aligned}
$$

# Marginal probability for hidden units

A similar derivation yields the marginal probability of the hidden units

$$p_{BB}(\boldsymbol{h}, \boldsymbol{\Theta}) = \frac{1}{Z_{BB}(\boldsymbol{\Theta})} e^{\boldsymbol{b}^T \boldsymbol{h}} \prod_i^M (1 + e^{a_i + \boldsymbol{w}_{i*}^T \boldsymbol{h}}).$$

# Conditional Probability Density Functions

We derive the probability of the hidden units given the visible units using Bayes' rule (we drop the explicit $\boldsymbol{\Theta}$ dependence)

$$
\begin{aligned}
p_{BB}(\boldsymbol{h}|\boldsymbol{x}) &= \frac{p_{BB}(\boldsymbol{x}, \boldsymbol{h})}{p_{BB}(\boldsymbol{x})} \\
&= \frac{\frac{1}{Z_{BB}} e^{\boldsymbol{a}^T \boldsymbol{x} + \boldsymbol{b}^T \boldsymbol{h} + \boldsymbol{x}^T \boldsymbol{W} \boldsymbol{h}}}{\frac{1}{Z_{BB}} e^{\boldsymbol{a}^T \boldsymbol{x}} \prod_j^N (1 + e^{b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}})} \\
&= \frac{e^{\boldsymbol{a}^T \boldsymbol{x}} e^{\sum_j^N (b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}) h_j}}{e^{\boldsymbol{a}^T \boldsymbol{x}} \prod_j^N (1 + e^{b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}})} \\
&= \prod_j^N \frac{e^{(b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}) h_j}}{1 + e^{b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}}} \\
&= \prod_j^N p_{BB}(h_j|\boldsymbol{x}).
\end{aligned}
$$

# On and off probabilities

From this we find the probability of a hidden unit being "on" or "off":

$$
\begin{aligned}
p_{BB}(h_j = 1|\boldsymbol{x}) =& \frac{e^{(b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j})h_j}}{1 + e^{b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}}} \\
=& \frac{e^{(b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j})}}{1 + e^{b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}}} \\
=& \frac{1}{1 + e^{-(b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j})}},
\end{aligned}
$$

and

$$
p_{BB}(h_j = 0|\boldsymbol{x}) = \frac{1}{1 + e^{b_j + \boldsymbol{x}^T \boldsymbol{w}_{*j}}}.
$$

# Conditional probability for visible units

Similarly we have that the conditional probability of the visible units given the hidden are

$$
\begin{aligned}
p_{BB}(\boldsymbol{x}|\boldsymbol{h}) &= \prod_{i}^{M} \frac{e^{(a_i + \boldsymbol{w}_{i*}^T \boldsymbol{h}) x_i}}{1 + e^{a_i + \boldsymbol{w}_{i*}^T \boldsymbol{h}}} \\
&= \prod_{i}^{M} p_{BB}(x_i|\boldsymbol{h}).
\end{aligned}
$$

We have

$$
\begin{aligned}
p_{BB}(x_i = 1|\boldsymbol{h}) &= \frac{1}{1 + e^{-(a_i + \boldsymbol{w}_{i*}^T \boldsymbol{h})}} \\
p_{BB}(x_i = 0|\boldsymbol{h}) &= \frac{1}{1 + e^{a_i + \boldsymbol{w}_{i*}^T \boldsymbol{h}}}.
\end{aligned}
$$

# Gaussian-Binary Restricted Boltzmann Machines

Inserting into the expression for $E_{RBM}(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta})$ in equation results in the energy

$$
\begin{aligned}
E_{GB}(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta}) = & \sum_{i}^{M} \frac{(x_i - a_i)^2}{2\sigma_i^2} - \sum_{j}^{N} b_j h_j - \sum_{ij}^{M,N} \frac{x_i w_{ij} h_j}{\sigma_i^2} \\
= & ||\frac{\boldsymbol{x} - \boldsymbol{a}}{2\boldsymbol{\sigma}}||^2 - \boldsymbol{b}^T \boldsymbol{h} - (\frac{\boldsymbol{x}}{\boldsymbol{\sigma}^2})^T \boldsymbol{W}\boldsymbol{h}.
\end{aligned}
$$

# Joint Probability Density Function

$$
\begin{aligned}
p_{GB}(\boldsymbol{x}, \boldsymbol{h}, \boldsymbol{\Theta}) &= \frac{1}{Z_{GB}} e^{-||\frac{\boldsymbol{x}-\boldsymbol{a}}{2\boldsymbol{\sigma}}||^2 + \boldsymbol{b}^T \boldsymbol{h} + (\frac{\boldsymbol{x}}{\boldsymbol{\sigma}^2})^T \boldsymbol{W}\boldsymbol{h}} \\
&= \frac{1}{Z_{GB}} e^{-\sum_i^M \frac{(x_i - a_i)^2}{2\sigma_i^2} + \sum_j^N b_j h_j + \sum_{ij}^{M,N} \frac{x_i w_{ij} h_j}{\sigma_i^2}} \\
&= \frac{1}{Z_{GB}} \prod_{ij}^{M,N} e^{-\frac{(x_i - a_i)^2}{2\sigma_i^2} + b_j h_j + \frac{x_i w_{ij} h_j}{\sigma_i^2}} .
\end{aligned}
$$

# Partition function

The partition function is given by

$$Z_{GB} = \int \sum_{\tilde{\boldsymbol{h}}}^{\tilde{\boldsymbol{H}}} e^{-||\frac{\tilde{\boldsymbol{x}}-\boldsymbol{a}}{2\sigma}||^2 + \boldsymbol{b}^T \tilde{\boldsymbol{h}} + (\frac{\tilde{\boldsymbol{x}}}{\sigma^2})^T \boldsymbol{W} \tilde{\boldsymbol{h}}} d\tilde{\boldsymbol{x}}.$$

# Marginal Probability Density Functions

We proceed to find the marginal probability densitites of the Gaussian-binary RBM. We first marginalize over the binary hidden units to find $p_{GB}(\boldsymbol{x})$

$$
\begin{aligned}
p_{GB}(\boldsymbol{x}) &= \sum_{\tilde{\boldsymbol{h}}}^{\tilde{\boldsymbol{H}}} p_{GB}(\boldsymbol{x}, \tilde{\boldsymbol{h}}) \\
&= \frac{1}{Z_{GB}} \sum_{\tilde{\boldsymbol{h}}}^{\tilde{\boldsymbol{H}}} e^{-||\frac{\boldsymbol{x}-\boldsymbol{a}}{2\boldsymbol{\sigma}}||^2 + \boldsymbol{b}^T \tilde{\boldsymbol{h}} + (\frac{\boldsymbol{x}}{\boldsymbol{\sigma}^2})^T \boldsymbol{W} \tilde{\boldsymbol{h}}} \\
&= \frac{1}{Z_{GB}} e^{-||\frac{\boldsymbol{x}-\boldsymbol{a}}{2\boldsymbol{\sigma}}||^2} \prod_{j}^{N} (1 + e^{b_j + (\frac{\boldsymbol{x}}{\boldsymbol{\sigma}^2})^T \boldsymbol{w}_{*j}}).
\end{aligned}
$$

# Then the visible units

We next marginalize over the visible units. This is the first time we marginalize over continuous values. We rewrite the exponential factor dependent on $\boldsymbol{x}$ as a Gaussian function before we integrate in the last step.

$$
\begin{aligned}
p_{GB}(\boldsymbol{h}) &= \int p_{GB}(\tilde{\boldsymbol{x}}, \boldsymbol{h}) d\tilde{\boldsymbol{x}} \\
&= \frac{1}{Z_{GB}} \int e^{-||\frac{\tilde{\boldsymbol{x}}-\boldsymbol{a}}{2\boldsymbol{\sigma}}||^2 + \boldsymbol{b}^T\boldsymbol{h} + (\frac{\tilde{\boldsymbol{x}}}{\boldsymbol{\sigma}^2})^T \boldsymbol{W}\boldsymbol{h}} d\tilde{\boldsymbol{x}} \\
&= \frac{1}{Z_{GB}} e^{\boldsymbol{b}^T\boldsymbol{h}} \int \prod_i^M e^{-\frac{(\tilde{x}_i - a_i)^2}{2\sigma_i^2} + \frac{\tilde{x}_i \boldsymbol{w}_{i*}^T \boldsymbol{h}}{\sigma_i^2}} d\tilde{\boldsymbol{x}} \\
&= \frac{1}{Z_{GB}} e^{\boldsymbol{b}^T\boldsymbol{h}} \left( \int e^{-\frac{(\tilde{x}_1 - a_1)^2}{2\sigma_1^2} + \frac{\tilde{x}_1 \boldsymbol{w}_{1*}^T \boldsymbol{h}}{\sigma_1^2}} d\tilde{x}_1 \right. \\
&\quad \times \int e^{-\frac{(\tilde{x}_2 - a_2)^2}{2\sigma_2^2} + \frac{\tilde{x}_2 \boldsymbol{w}_{2*}^T \boldsymbol{h}}{\sigma_2^2}} d\tilde{x}_2 \\
&\quad \times \ldots
\end{aligned}
$$

# Conditional Probability Density Functions

We finish by deriving the conditional probabilities.

$$
\begin{aligned}
p_{GB}(\boldsymbol{h}|\boldsymbol{x}) &= \frac{p_{GB}(\boldsymbol{x}, \boldsymbol{h})}{p_{GB}(\boldsymbol{x})} \\
&= \frac{\frac{1}{Z_{GB}} e^{-||\frac{\boldsymbol{x}-\boldsymbol{a}}{2\boldsymbol{\sigma}}||^2 + \boldsymbol{b}^T \boldsymbol{h} + (\frac{\boldsymbol{x}}{\boldsymbol{\sigma}^2})^T \boldsymbol{W}\boldsymbol{h}}}{\frac{1}{Z_{GB}} e^{-||\frac{\boldsymbol{x}-\boldsymbol{a}}{2\boldsymbol{\sigma}}||^2} \prod_j^N (1 + e^{b_j + (\frac{\boldsymbol{x}}{\boldsymbol{\sigma}^2})^T \boldsymbol{w}_{*j}})} \\
&= \prod_j^N \frac{e^{(b_j + (\frac{\boldsymbol{x}}{\boldsymbol{\sigma}^2})^T \boldsymbol{w}_{*j}) h_j}}{1 + e^{b_j + (\frac{\boldsymbol{x}}{\boldsymbol{\sigma}^2})^T \boldsymbol{w}_{*j}}} \\
&= \prod_j^N p_{GB}(h_j|\boldsymbol{x}).
\end{aligned}
$$

# Hidden units

The conditional probability of a binary hidden unit $h_j$ being on or off again takes the form of a sigmoid function

$$
\begin{aligned}
p_{GB}(h_j = 1|\boldsymbol{x}) &= \frac{e^{b_j + (\frac{\boldsymbol{x}}{\sigma^2})^T \boldsymbol{w}_{*j}}}{1 + e^{b_j + (\frac{\boldsymbol{x}}{\sigma^2})^T \boldsymbol{w}_{*j}}} \\
&= \frac{1}{1 + e^{-b_j - (\frac{\boldsymbol{x}}{\sigma^2})^T \boldsymbol{w}_{*j}}} \\
p_{GB}(h_j = 0|\boldsymbol{x}) &= \frac{1}{1 + e^{b_j + (\frac{\boldsymbol{x}}{\sigma^2})^T \boldsymbol{w}_{*j}}}.
\end{aligned}
$$

# Visible units

The conditional probability of the continuous $\boldsymbol{x}$ now has another form, however.

$$
\begin{aligned}
p_{GB}(\boldsymbol{x}|\boldsymbol{h}) &= \frac{p_{GB}(\boldsymbol{x},\boldsymbol{h})}{p_{GB}(\boldsymbol{h})} \\
&= \frac{\frac{1}{Z_{GB}} e^{-||\frac{\boldsymbol{x}-\boldsymbol{a}}{2\boldsymbol{\sigma}}||^2 + \boldsymbol{b}^T\boldsymbol{h} + (\frac{\boldsymbol{x}}{\boldsymbol{\sigma}^2})^T \boldsymbol{W}\boldsymbol{h}}}{\frac{1}{Z_{GB}} e^{\boldsymbol{b}^T\boldsymbol{h}} \prod_i^M \sqrt{2\pi\sigma_i^2} e^{\frac{2a_i \boldsymbol{w}_{i*}^T\boldsymbol{h} + (\boldsymbol{w}_{i*}^T\boldsymbol{h})^2}{2\sigma_i^2}}} \\
&= \prod_i^M \frac{1}{\sqrt{2\pi\sigma_i^2}} \frac{e^{-\frac{(x_i-a_i)^2}{2\sigma_i^2} + \frac{x_i \boldsymbol{w}_{i*}^T\boldsymbol{h}}{2\sigma_i^2}}}{e^{\frac{2a_i \boldsymbol{w}_{i*}^T\boldsymbol{h} + (\boldsymbol{w}_{i*}^T\boldsymbol{h})^2}{2\sigma_i^2}}} \\
&= \prod_i^M \frac{1}{\sqrt{2\pi\sigma_i^2}} \frac{e^{-\frac{x_i^2 - 2a_i x_i + a_i^2 - 2x_i \boldsymbol{w}_{i*}^T\boldsymbol{h}}{2\sigma_i^2}}}{e^{\frac{2a_i \boldsymbol{w}_{i*}^T\boldsymbol{h} + (\boldsymbol{w}_{i*}^T\boldsymbol{h})^2}{2\sigma_i^2}}} \\
&= \prod_i^M \frac{1}{\sqrt{2\pi\sigma_i^2}} e^{-\frac{x_i^2 - 2a_i x_i + a_i^2 - 2x_i \boldsymbol{w}_{i*}^T\boldsymbol{h} + 2a_i \boldsymbol{w}_{i*}^T\boldsymbol{h} + (\boldsymbol{w}_{i*}^T\boldsymbol{h})^2}{2\sigma_i^2}}
\end{aligned}
$$

# Comments

The form of these conditional probabilities explains the name "Gaussian" and the form of the Gaussian-binary energy function. We see that the conditional probability of $x_i$ given $\boldsymbol{h}$ is a normal distribution with mean $b_i + \boldsymbol{w}_{i*}^T \boldsymbol{h}$ and variance $\sigma_i^2$.

# Energy-based models and Langevin sampling

See discussions in Foster, chapter 7 on energy-based models at https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition/tree/main/notebooks/07_ebm/01_ebm
That notebook is based on a recent article by Du and Mordatch, **Implicit generation and modeling with energy-based models**, see https://arxiv.org/pdf/1903.08689.pdf.

# Tensor-flow examples

1. To create Boltzmann machine using Keras, see Babcock and Bali chapter 4, see https://github.com/PacktPublishing/Hands-On-Generative-AI-with-Python-and-TensorFlow-2/blob/master/Chapter_4/models/rbm.py
2. See also Foster, chapter 7 on energy-based models at https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition/tree/main/notebooks/07_ebm/01_ebm

# Theory of Variational Autoencoders

Let us remind ourself about what an autoencoder is, see the jupyter-notebooks at https://github.com/CompPhysics/AdvancedMachineLearning/blob/main/doc/pub/week8/ipynb/week8.ipynb and https://github.com/CompPhysics/AdvancedMachineLearning/blob/main/doc/pub/week9/ipynb/week9.ipynb.

# The Autoencoder again

Autoencoders are neural networks where the outputs are its own inputs. They are split into an **encoder part** which maps the input $\boldsymbol{x}$ via a function $f(\boldsymbol{x}, \boldsymbol{W})$ (this is the encoder part) to a **so-called code part** (or intermediate part) with the result $\boldsymbol{h}$

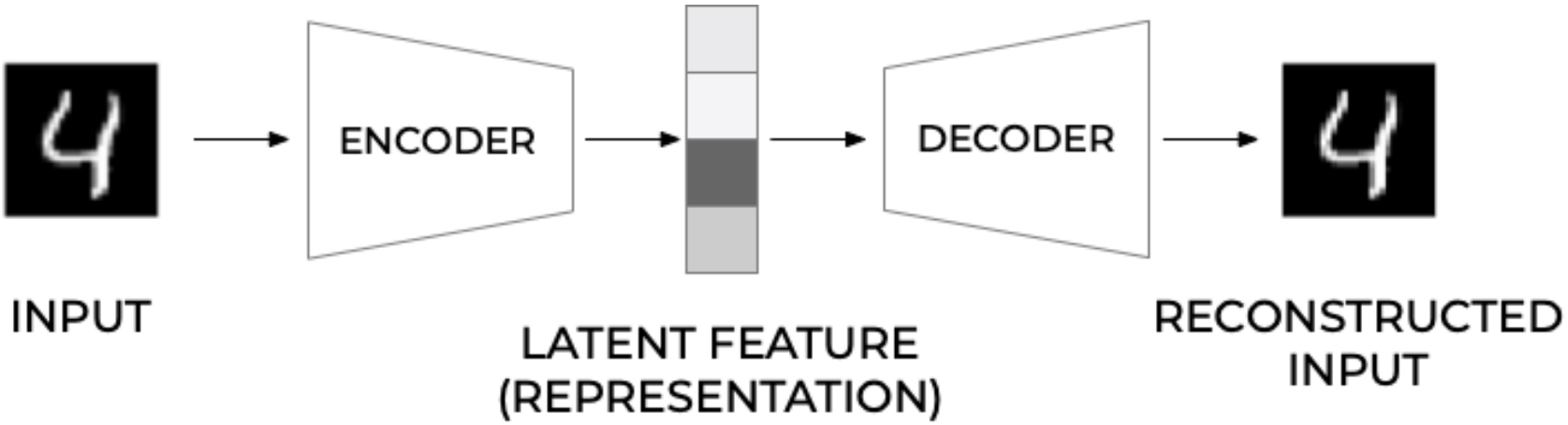
$$\boldsymbol{h} = f(\boldsymbol{x}, \boldsymbol{W})),$$

where $\boldsymbol{W}$ are the weights to be determined. The **decoder** parts maps, via its own parameters (weights given by the matrix $\boldsymbol{V}$ and its own biases) to the final ouput

$$\tilde{\boldsymbol{x}} = g(\boldsymbol{h}, \boldsymbol{V})).$$

The goal is to minimize the construction error, often done by optimizing the means squared error.

# Schematic image of an Autoencoder



# Mathematics of Variational Autoencoders

We have defined earlier a probability (marginal) distribution with hidden variables $\boldsymbol{h}$ and parameters $\boldsymbol{\Theta}$ as

$$p(\boldsymbol{x};\boldsymbol{\Theta}) = \int d\boldsymbol{h} p(\boldsymbol{x},\boldsymbol{h};\boldsymbol{\Theta}),$$

for continuous variables $\boldsymbol{h}$ and

$$p(\boldsymbol{x};\boldsymbol{\Theta}) = \sum_{\boldsymbol{h}} p(\boldsymbol{x},\boldsymbol{h};\boldsymbol{\Theta}),$$

for discrete stochastic events $\boldsymbol{h}$. The variables $\boldsymbol{h}$ are normally called the **latent variables** in the theory of autoencoders. We will also call then for that here.

# Using the conditional probability

Using the the definition of the conditional probabilities $p(\boldsymbol{x}|\boldsymbol{h};\boldsymbol{\Theta})$, $p(\boldsymbol{h}|\boldsymbol{x};\boldsymbol{\Theta})$ and and the prior $p(\boldsymbol{h})$, we can rewrite the above equation as

$$p(\boldsymbol{x};\boldsymbol{\Theta}) = \sum_{\boldsymbol{h}} p(\boldsymbol{x}|\boldsymbol{h};\boldsymbol{\Theta})p(\boldsymbol{h}),$$

which allows us to make the dependence of $\boldsymbol{x}$ on $\boldsymbol{h}$ explicit by using the law of total probability. The intuition behind this approach for finding the marginal probability for $\boldsymbol{x}$ is to optimize the above equations with respect to the parameters $\boldsymbol{\Theta}$. This is done normally by maximizing the probability, the so-called maximum-likelihood approach discussed earlier.

# VAEs versus autoencoders

This trained probability is assumed to be able to produce similar samples as the input. In VAEs it is then common to compare via for example the mean-squared error or the cross-entropy the predicted values with the input values. Compared with autoencoders, we are now producing a probability instead of a functions which mimicks the input.
In VAEs, the choice of this output distribution is often Gaussian, meaning that the conditional probability is

$$p(\boldsymbol{x}|\boldsymbol{h};\boldsymbol{\Theta}) = N(\boldsymbol{x}|f(\boldsymbol{h};\boldsymbol{\Theta}), \sigma^2 \times \boldsymbol{I}),$$

with mean value given by the function $f(\boldsymbol{h};\boldsymbol{\Theta})$ and a diagonal covariance matrix multiplied by a parameter $\sigma^2$ which is treated as a hyperparameter.

# Gradient descent

By having a Gaussian distribution, we can use gradient descent (or any other optimization technique) to increase $p(\boldsymbol{x}; \boldsymbol{\Theta})$ by making $f(\boldsymbol{h}; \boldsymbol{\Theta})$ approach $\boldsymbol{x}$ for some $\boldsymbol{h}$, gradually making the training data more likely under the generative model. The important property is simply that the marginal probability can be computed, and it is continuous in $\boldsymbol{\Theta}$.

# Are VAEs just modified autoencoders?

The mathematical basis of VAEs actually has relatively little to do with classical autoencoders, for example the sparse autoencoders or denoising autoencoders discussed earlier.
VAEs approximately maximize the probability equation discussed above. They are called autoencoders only because the final training objective that derives from this setup does have an encoder and a decoder, and resembles a traditional autoencoder. Unlike sparse autoencoders, there are generally no tuning parameters analogous to the sparsity penalties. And unlike sparse and denoising autoencoders, we can sample directly from $p(\boldsymbol{x})$ without performing Markov Chain Monte Carlo.

# Training VAEs

To solve the integral or sum for $p(\boldsymbol{x})$, there are two problems that VAEs must deal with: how to define the latent variables $\boldsymbol{h}$, that is decide what information they represent, and how to deal with the integral over $\boldsymbol{h}$. VAEs give a definite answer to both.

## Motivation from Kingma and Welling, An Introduction to Variational Autoencoders, https://arxiv.org/abs/1906.02691

*There are many reasons why generative modeling is attractive. First, we can express physical laws and constraints into the generative process while details that we don't know or care about, i.e. nuisance variables, are treated as noise. The resulting models are usually highly intuitive and interpretable and by testing them against observations we can confirm or reject our theories about how the world works. Another reason for trying to understand the generative process of data is that it naturally expresses causal relations of the world. Causal relations have the great advantage that they generalize much better to new situations than mere correlations. For instance, once we understand the generative process of an earthquake, we can use that knowledge both in California and in Chile.*

# Mathematics of VAEs

We want to train the marginal probability with some latent varrables $\boldsymbol{h}$

$$p(\boldsymbol{x}; \boldsymbol{\Theta}) = \int d\boldsymbol{h} p(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}),$$

for the continuous version (see previous slides for the discrete variant).

# Using the KL divergence

In practice, for most $\boldsymbol{h}$, $p(\boldsymbol{x}|\boldsymbol{h};\boldsymbol{\Theta})$ will be nearly zero, and hence contributes almost nothing to our estimate of $p(\boldsymbol{x})$.
The key idea behind the variational autoencoder is to attempt to sample values of $\boldsymbol{h}$ that are likely to have produced $\boldsymbol{x}$, and compute $p(\boldsymbol{x})$ just from those.
This means that we need a new function $Q(\boldsymbol{h}|\boldsymbol{x})$ which can take a value of $\boldsymbol{x}$ and give us a distribution over $\boldsymbol{h}$ values that are likely to produce $\boldsymbol{x}$. Hopefully the space of $\boldsymbol{h}$ values that are likely under $Q$ will be much smaller than the space of all $\boldsymbol{h}$'s that are likely under the prior $p(\boldsymbol{h})$. This lets us, for example, compute $E_{\boldsymbol{h}\sim Q}p(\boldsymbol{x}|\boldsymbol{h})$ relatively easily. Note that we drop $\boldsymbol{\Theta}$ from here and for notational simplicity.

# Kullback-Leibler again

However, if $\boldsymbol{h}$ is sampled from an arbitrary distribution with PDF $Q(\boldsymbol{h})$, which is not $\mathcal{N}(0, I)$, then how does that help us optimize $p(\boldsymbol{x})$?

The first thing we need to do is relate $E_{\boldsymbol{h}\sim Q} P(\boldsymbol{x}|\boldsymbol{h})$ and $p(\boldsymbol{x})$. We will see where $Q$ comes from later.

The relationship between $E_{\boldsymbol{h}\sim Q} p(\boldsymbol{x}|\boldsymbol{h})$ and $p(\boldsymbol{x})$ is one of the cornerstones of variational Bayesian methods. We begin with the definition of Kullback-Leibler divergence (KL divergence or $\mathcal{D}$) between $p(\boldsymbol{h}|\boldsymbol{x})$ and $Q(\boldsymbol{h})$, for some arbitrary $Q$ (which may or may not depend on $\boldsymbol{x}$):

$$\mathcal{D}\left[Q(\boldsymbol{h}) \| p(\boldsymbol{h}|\boldsymbol{x})\right] = E_{\boldsymbol{h}\sim Q}\left[\log Q(\boldsymbol{h}) - \log p(\boldsymbol{h}|\boldsymbol{x})\right].$$

# And applying Bayes rule

We can get both $p(\boldsymbol{x})$ and $p(\boldsymbol{x}|\boldsymbol{h})$ into this equation by applying Bayes rule to $p(\boldsymbol{h}|\boldsymbol{x})$

$$\mathcal{D}\left[Q(\boldsymbol{h})\|p(\boldsymbol{h}|\boldsymbol{x})\right] = E_{\boldsymbol{h}\sim Q}\left[\log Q(\boldsymbol{h}) - \log p(\boldsymbol{x}|\boldsymbol{h}) - \log p(\boldsymbol{h})\right] + \log p(\boldsymbol{x}).$$

Here, $\log p(\boldsymbol{x})$ comes out of the expectation because it does not depend on $\boldsymbol{h}$. Negating both sides, rearranging, and contracting part of $E_{\boldsymbol{h}\sim Q}$ into a KL-divergence terms yields:

$$\log p(\boldsymbol{x}) - \mathcal{D}\left[Q(\boldsymbol{h})\|p(\boldsymbol{h}|\boldsymbol{x})\right] = E_{\boldsymbol{h}\sim Q}\left[\log p(\boldsymbol{x}|\boldsymbol{h})\right] - \mathcal{D}\left[Q(\boldsymbol{h})\|P(\boldsymbol{h})\right].$$

# Rearranging

Using Bayes rule we obtain

$$E_{\boldsymbol{h}\sim Q}\left[\log p(y_i|\boldsymbol{h}, x_i)\right] = E_{\boldsymbol{h}\sim Q}\left[\log p(\boldsymbol{h}|y_i, x_i) - \log p(\boldsymbol{h}|x_i) + \log p(y_i|x_i)\right]$$

Rearranging the terms and subtracting $E_{\boldsymbol{h}\sim Q} \log Q(\boldsymbol{h})$ from both sides gives

$$\log P(y_i|x_i) - E_{\boldsymbol{h}\sim Q}\left[\log Q(\boldsymbol{h}) - \log p(\boldsymbol{h}|x_i, y_i)\right] = \\ E_{\boldsymbol{h}\sim Q}\left[\log p(y_i|\boldsymbol{h}, x_i) + \log p(\boldsymbol{h}|x_i) - \log Q(\boldsymbol{h})\right]$$

Note that $\boldsymbol{x}$ is fixed, and $Q$ can be *any* distribution, not just a distribution which does a good job mapping $\boldsymbol{x}$ to the $\boldsymbol{h}$'s that can produce $X$.

# Inferring the probability

Since we are interested in inferring $p(\boldsymbol{x})$, it makes sense to construct a $Q$ which *does* depend on $\boldsymbol{x}$, and in particular, one which makes $\mathcal{D}\left[Q(\boldsymbol{h}) \| p(\boldsymbol{h}|\boldsymbol{x})\right]$ small

$$\log p(\boldsymbol{x}) - \mathcal{D}\left[Q(\boldsymbol{h}|\boldsymbol{x}) \| p(\boldsymbol{h}|\boldsymbol{x})\right] = E_{\boldsymbol{h}\sim Q}\left[\log p(\boldsymbol{x}|\boldsymbol{h})\right] - \mathcal{D}\left[Q(\boldsymbol{h}|\boldsymbol{x}) \| p(\boldsymbol{h})\right].$$

Hence, during training, it makes sense to choose a $Q$ which will make $E_{\boldsymbol{h}\sim Q}[\log Q(\boldsymbol{h}) - \log p(\boldsymbol{h}|x_i, y_i)]$ (a $\mathcal{D}$-divergence) small, such that the right hand side is a close approximation to $\log p(y_i|y_i)$.

# Central equation of VAEs

This equation serves as the core of the variational autoencoder, and it is worth spending some time thinking about what it means.

1. The left hand side has the quantity we want to maximize, namely $\log p(\boldsymbol{x})$ plus an error term.
2. The right hand side is something we can optimize via stochastic gradient descent given the right choice of $Q$.